KFANDRA Helper — Developer Guide

Technical administration for the developer: viewing/editing the database, changing point values, managing players, environment variables, and deploying. Needs the Supabase and Vercel logins. Keep this internal.

> For the non-technical, in-app Admin section (viewing submissions), see **admin-guide.md**.

> **Never paste secrets anywhere.** The Supabase `service_role` key, the database password, and `SESSION_SECRET` are secrets. This guide tells you *where* they live, never *what* they are.

1. What the app is

- **Frontend:** a Next.js app (a mobile-first PWA) hosted on **Vercel**.
- **Backend/data: Supabase** (hosted PostgreSQL + the service that serves it).
- **Auth:** custom **phone + 4-digit PIN** (not Supabase Auth). A signed cookie keeps players logged in for 60 days.
- **Three player modes: MMG** (session points), **Gym** (daily logging), **Diet** (daily food logging).

Golden rule: all game rules are *data*, not code. Scoring values, game types, and the food/gym catalogues live in database tables you can edit at runtime — no code change or redeploy needed for those.

Thing	Where
Live app	https://kfandra-track.vercel.app
Supabase project ref	`gcsgctfbgpihxmqqlxch`
Supabase URL	`https://gcsgctfbgpihxmqqlxch.supabase.co`
Hosting	Vercel project `kfandra-track`
Code repo	`github.com/jaidevk/kfandra-track`

2. Viewing & editing the database

Hosted (production) — the normal way

1. Go to the **Supabase dashboard** → open the project (`gcsgctfbgpihxmqqlxch`).
2. **Table Editor** (left sidebar) — browse and edit any table like a spreadsheet. Click a row to edit, use **Insert** to add rows.
3. **SQL Editor** — run queries for anything bulk or precise (examples below).

You sign in to the Supabase dashboard with your own Supabase account that has access to the project. (Ask the project owner to invite you if you don't.)

Local (for testing changes safely)

If you have the code checked out and Docker running:

```
npx supabase start        # boots a local Postgres + Studio
# Studio opens at http://127.0.0.1:54323
npx supabase stop         # when done
```

The local database is a throwaway copy — a safe sandbox to try edits before touching production.

---

3. The tables that matter

Table	What it holds
`players`	Members: name, phone, role, active flag, hashed PIN
`sessions`	Training sessions (auto-created for Tue/Thu/Sat)
`point_rules`	**All MMG scoring values** (result, stats, participation, order)
`game_types`	The list of game types (Football short, Rugby, Fooba, …)
`app_config`	Generic key/value settings
`gym_catalog`	Gym body parts / exercises

Table	What it holds
`meal_slots`	The 8 diet meal slots (name, time window, emoji)
`food_catalog`	The food list players tap to log
`mmg_entries`, `submission_games`, `submission_game_stats`, `submission_others`	A player's submitted MMG data
`gym_logs`, `gym_log_exercises`	Gym entries
`diet_logs`, `diet_log_meals`, `diet_log_items`	Diet entries

4. Common tasks

4a. Manage players & roles

Open `players` in the Table Editor.

- **Roles** (column `role`): `super_admin` > `kfandra` > `admin` > `user`. The first three are "staff" and unlock elevated database access via row-level security. New registrations default to `user`. To promote someone, change their `role`. The **`kfandra`** role is the head coach / club account: it has staff access **and** is excluded from MMG order points (KFANDRA runs the sessions and doesn't compete, so it never counts toward arrival/confirmation points or the `N` denominator).
- **Disable an account:** set `is_active` to `false`. They stay in the DB but can no longer log in (and are logged out on their next page load). Re-enable by setting it back to `true`. Prefer this over deleting.
- **Phone** is the unique login identifier (stored as `+91XXXXXXXXXX`).
- **PIN resets:** there is no self-service "forgot PIN" yet. If a player is locked out, the practical fix today is to delete their `players` row so they can re-register with the same phone. (A proper reset flow is future work.)

> You never see or set the raw PIN — only a one-way hash is stored.

4b. Edit MMG scoring (the most common request)

All point values live in `point_rules`. Edit the `points` column of a row and it takes effect immediately the next time a player's page computes their score — no redeploy.

`point_rules.scope` tells you what a row controls:

- `result` — points per game outcome. Defaults: `won` 1000, `drew` 100, `lost` 0. (A player tallies how many games they won/drew/lost; points multiply by the count.)
- `stat` — points per recorded stat. Defaults include: goals 500, tries 500, assists 200, pre-assists 100, saves 500, goal-line saves 500, rebound wall 500, tackle 200; penalties are negative: yellow –100, red –500, blue –1000, late challenge –100, foul –200.
- `participation` — one-off session bonuses: GWW unpacking 500, GWW packing 500, session packing (PTM) 200, confirmed availability 500.
- `order` — `base_per_rank` (100) is the unit for the order-of-arrival ladder.

Per-game-type overrides: a `point_rules` row with a non-null `game_type_id` applies *only* to that game type. Examples already seeded:

- Rugby **tackle** is worth 500 (override) instead of the default 200.
- **Goal conceded** (–200) applies *only* to Fooba (Big Goal).

To change a value: edit the matching row's `points`. To add a new override: insert a row with the same `rule_key`, the `points` you want, and the `game_type_id` of the target type (copy the id from `game_types`).

4c. Manage game types

Edit `game_types`: `name`, `emoji`, `sort_order` (display order), `is_active`. Adding a new active type makes it appear in the MMG game editor automatically. (Stats available per type are defined in app config/code — ping the developer if a new type needs a bespoke stat set.)

4d. Manage the Diet catalogue

- `meal_slots` — the meal rows players see (name, time window, emoji, order).
- `food_catalog` — the tappable foods, grouped by a section label, with a unit (e.g. "1 roti"). Edit/add rows here to change what players can log.

4e. Manage the Gym catalogue

Edit `gym_catalog` to change the body parts / exercises players choose from.

4f. Sessions

You do **not** create sessions by hand. The app treats every **Tue/Thu/Sat** as a session and creates the `sessions` row on demand the first time someone logs for that date. Players pick the date from a dropdown in MMG.

---

5. Changing the schema (developer task)

Adding columns/tables is **not** done in the dashboard — it's done as a migration in the codebase so every environment stays in sync. The flow:

1. A developer adds a SQL file under `supabase/migrations/`.
2. It's reviewed and merged (push to `main`).
3. Someone with the DB password applies it to production:

```
npx supabase db push      # prompts for the DB password; you type it in
```

> The **code** auto-deploys on push, but **database migrations are applied manually**. After shipping a change that needs new columns, run `db push` promptly or the new code can error against the old schema.

6. Environment variables (Vercel)

Set under Vercel → project `kfandra-track` → **Settings** → **Environment Variables** (Production scope). After changing any of these you must **redeploy** for it to take effect (`vercel redeploy kfandra-track.vercel.app` or the dashboard's Redeploy button).

Variable	Purpose	Secret?
`NEXT_PUBLIC_SUPABASE_URL`	Which Supabase project	No
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public client key	Low
`SUPABASE_SERVICE_ROLE_KEY`	Server admin key (bypasses RLS)	**YES**
`SESSION_SECRET`	Signs login cookies	**YES**
`NEXT_PUBLIC_POSTHOG_PROJECT_TOKEN`	Product analytics (optional)	Low
`NEXT_PUBLIC_POSTHOG_HOST`	PostHog host (optional)	No

Rotating Supabase keys: copy the value from Supabase → Project Settings → API, paste it into the matching Vercel variable, save, redeploy. The `service_role` key and `anon` key must both come from the *same* project as the URL — a mismatch causes "Invalid API key" errors on every data operation.

7. Deploying

- Push to `main` → GitHub Actions runs the quality gate (lint, tests, build) **and** Vercel auto-deploys to production.
- Pull requests get their own preview deployment.
- You normally never deploy by hand; just merge to `main`.

---

8. Observability (knowing what's happening)

- **Errors / server logs:** Vercel → project → **Logs** (Functions). Server-side failures are logged with a `[jacaranda:error]` tag plus the underlying cause, so problems are diagnosable instead of silent.
- **Product analytics:** PostHog (once `NEXT_PUBLIC_POSTHOG_PROJECT_TOKEN` is set). Shows page views, funnels (landing → register → mode usage), and session replays to find where players get stuck. Players are keyed by an anonymous id — **no phone numbers or PINs** are sent to analytics.

---

9. Security notes

- Players authenticate with phone + a 4-digit PIN; only a hash is stored.
- The browser uses a restricted key; all privileged data access happens server-side with the `service_role` key, which **bypasses** row-level security. Keep that key secret.
- Row-Level Security is enabled on the tables; staff roles get broader read access via the role hierarchy.
- Don't put personal data in URLs, and don't share the `service_role` key, DB password, or `SESSION_SECRET` over chat/email.

---

10. Quick "where do I…" index

I want to…	Do this
Change how many points a goal is worth	Edit `point_rules` row (`stat` / `goals`) → `points`
Change win/draw points	Edit `point_rules` rows with scope `result`

I want to…	Do this
Make a stat worth more for one game type	Add a `point_rules` row with that `game_type_id`
Add/disable a game type	Edit `game_types` (`is_active` , `sort_order`)
Add a food players can log	Insert into `food_catalog`
Promote someone to admin	Set their `players.role`
Lock someone out	Set `players.is_active = false`
See why something errored	Vercel → Logs (look for `[jacaranda:error]`)
Apply a schema change	Developer merges a migration, then `npx supabase db push`
Change the words on a screen	Edit `src/content/strings.ts` , commit, push (§11)

11. Editing the words on screen (display strings)

The headings, button labels, and small print on the **home** and **sign-in** screens are **not** hard-coded in among the program logic — they live in one plain file you can edit:

`src/content/strings.ts`

Open it and you'll see labelled text grouped by screen, e.g.:

```
brand: {
  appName: "The Jacaranda App",
  motto: "Respect, Trust, Integrity, Passion & Humility",
  ...
},
home: {
  mmg: { title: "MMG", subtitle: "Tap entries · per session" },
  ...
},
```

To change a word: edit only the text **inside the quotes**. Keep the quotes and the commas. Don't rename the labels on the left (e.g. `appName:`) — the app looks them up by those names. Then:

```
git add src/content/strings.ts
git commit -m "Reword the home screen"
git push          # the site rebuilds & redeploys automatically (a few minutes)
```

> **Why a file and not the database?** Screen copy is versioned with the code so a bad edit is easy to roll back, and a typo can't take the site down at runtime. The trade-off is that a change needs a commit + the automatic redeploy, not an instant Table-Editor save.

What lives where — the full config map

Not everything is in that strings file. Here's where each kind of setting lives:

You want to change…	Where	Takes effect
Screen headings / labels / button text	`src/content/strings.ts` (§11)	After commit + auto-redeploy
Points / scoring values	`point_rules` table (§4b)	Immediately (next score calc)
Game types	`game_types` table (§4c)	Immediately
Diet meal slots & food list	`meal_slots`, `food_catalog` (§4d)	Immediately
Gym exercises	`gym_catalog` (§4e)	Immediately
Server-shown login errors	`src/lib/auth/actions.ts`	After commit + auto-redeploy
Generic key/value settings	`app_config` table	Immediately

Rule of thumb: **lists and numbers** are database tables you edit live; **fixed screen wording** is the strings file you edit-and-redeploy.